FinTrack: Digital Wallet & Smart Finance Analytics Platform

1. Project Purpose

This document is designed to help a group of 5 computer science students align around a practical FinTech project, understand the scope, and split the work clearly.

The proposed project is:

> **FinTrack — A secure digital wallet and smart personal finance analytics platform.**

The system allows users to create digital wallets, make simulated transfers, track balances using a double-entry ledger, view transaction history, and receive spending insights through dashboards and alerts.

---

2. Why This Is a Good FinTech Project

This project is strong because it combines real-world financial concepts with achievable software engineering tasks.

It includes:

- User authentication
- Digital wallets
- Simulated payments
- Double-entry accounting principles
- Transaction history
- Budgeting and spending analytics
- Fraud or anomaly alerts
- Admin monitoring
- API design
- Database design
- Frontend dashboards
- Testing and deployment

It is realistic enough to feel like a banking or wallet product, but still manageable for a student team.

---

3. High-Level Project Description

FinTrack is a web application where users can:

1. Register and log in securely.
2. Create one or more digital wallets.
3. Add simulated funds to a wallet.

4. Transfer money to another user.
5. View wallet balances.
6. View transaction history.
7. Track spending by category.
8. Set monthly budgets.
9. Receive alerts for suspicious or unusual activity.
10. Use an admin panel to monitor users and transactions.

The core technical idea is that all financial movements are recorded using a **double-entry ledger**.

Example:

```
Alice sends €50 to Bob

Debit:  Alice Wallet  €50
Credit: Bob Wallet    €50
```

Every transaction must balance. This gives the project a serious FinTech foundation.

---

4. Project Goals

Functional Goals

By the end of the project, the system should allow users to:

- Create an account
- Log in securely
- Create a wallet
- View wallet balance
- Transfer simulated money to another user
- View transaction history
- Categorize transactions
- View analytics dashboards
- Set spending budgets
- Receive alerts for unusual transactions

Technical Goals

The team should demonstrate:

- Clean API design
- Good database modelling
- Secure authentication
- Reliable transaction processing

- Clear separation of responsibilities
- Good frontend user experience
- Testing and documentation
- Basic deployment using Docker or cloud hosting

---

5. Suggested Technology Stack

The exact stack can be adjusted based on what the students know.

Backend Options

Recommended:

- **C# ASP.NET Core Web API**

Alternatives:

- Java Spring Boot
- Node.js with NestJS or Express
- Python FastAPI

Frontend Options

Recommended:

- **React**

Alternatives:

- Angular
- Vue
- Blazor
- Flutter

Database Options

Recommended:

- **PostgreSQL** or **SQL Server**

Alternatives:

- MySQL
- SQLite for a simpler prototype

Other Tools

- GitHub for source control
- Docker for deployment
- Swagger/OpenAPI for API documentation
- JWT for authentication
- Chart.js or Recharts for dashboards
- Postman for API testing

---

6. Core Features

6.1 User Management

Users should be able to:

- Register
- Log in
- Log out
- View their profile
- Change password, optional

Security requirements:

- Passwords must be hashed
- JWT or session-based authentication should be used
- Users should only access their own wallets and transactions

---

6.2 Wallet Management

Each user can have one or more wallets.

Example wallets:

- Main Wallet
- Savings Wallet
- Travel Wallet

Wallet data should include:

- Wallet ID
- User ID
- Wallet name
- Currency
- Current balance

- Status: Active, Frozen, Closed
- Created date

6.3 Simulated Deposits

Users can add simulated funds to their wallet.

Example:

```
User deposits €500 into Main Wallet
```

This is not a real payment integration. It is only used to simulate money entering the system.

6.4 Transfers Between Users

A user should be able to send money to another user.

Example:

```
Alice sends €25 to Bob
```

The system should:

1. Check that Alice has enough balance.
2. Create a transaction record.
3. Create ledger postings.
4. Debit Alice's wallet.
5. Credit Bob's wallet.
6. Update balances.
7. Show the transaction in both users' histories.

6.5 Double-Entry Ledger

Every money movement should be stored as a transaction with at least two postings.

Example transfer:

Posting	Account	Debit	Credit
1	Alice Wallet	€50	
2	Bob Wallet		€50

The total debit amount must always equal the total credit amount.

This is one of the most important parts of the project because it shows proper financial system design.

6.6 Transaction History

Users should be able to see:

- Date and time
- Transaction type
- Amount
- Sender
- Receiver
- Status
- Description
- Category

Example transaction statuses:

- Pending
- Completed
- Failed
- Reversed

6.7 Budgeting

Users can set monthly budgets by category.

Example:

Category	Monthly Budget
Food	€300
Transport	€100
Entertainment	€150

The app should show progress against each budget.

Example alert:

```
You have used 85% of your Food budget this month.
```

6.8 Spending Analytics

The dashboard should show:

- Total income
- Total spending

- Wallet balance
- Spending by category
- Monthly spending trend
- Biggest transactions
- Recurring payments, optional

Suggested charts:

- Pie chart for spending by category
- Line chart for spending over time
- Bar chart for monthly budget usage

6.9 Alerts and Anomaly Detection

The system can generate simple alerts.

Examples:

- Large transaction detected
- Multiple transactions in a short time
- Spending is higher than usual
- Budget limit nearly reached
- Transaction to a new receiver

This can be rule-based rather than AI-based.

Example rules:

```
If transaction amount > €1,000 => Large Transaction Alert
If user spends more than 80% of budget => Budget Warning
If 5 transfers happen within 10 minutes => Suspicious Activity Alert
```

6.10 Admin Panel

An admin user can:

- View all users
- View all wallets
- View all transactions
- View suspicious alerts
- Freeze a wallet, optional
- Reverse a transaction, optional

This gives the project a banking operations feel.

---

7. Suggested Team Split for 5 Students

Student 1 — Backend API Lead

Main Responsibility

Design and implement the backend REST API.

Tasks

- Set up backend project
- Create API structure
- Implement authentication endpoints
- Implement user endpoints
- Implement wallet endpoints
- Implement transfer endpoints
- Add API validation
- Add Swagger documentation
- Coordinate API contracts with frontend

Main Deliverables

- Working backend API
- API documentation
- Authentication flow
- Transfer API
- Wallet API

Student 2 — Database & Ledger Lead

Main Responsibility

Design the database and implement the double-entry ledger model.

Tasks

- Design entity relationship diagram
- Create database schema
- Implement migrations
- Create tables for users, wallets, transactions, postings, budgets, and alerts
- Enforce double-entry transaction rules
- Ensure debit equals credit
- Handle balance updates safely

- Seed demo data

Main Deliverables

- Database schema
- ERD diagram
- Ledger model
- Balance calculation logic
- Demo dataset

Student 3 — Frontend & User Experience Lead

Main Responsibility

Build the user interface.

Tasks

- Set up frontend project
- Create login and registration screens
- Create dashboard page
- Create wallet page
- Create transfer page
- Create transaction history page
- Create budget page
- Connect frontend to backend APIs
- Make the UI clean and demo-friendly

Main Deliverables

- Working web interface
- Dashboard screens
- Wallet management screens
- Transfer screen
- Transaction history screen

Student 4 — Analytics, Budgeting & Alerts Lead

Main Responsibility

Build the analytics and intelligence layer.

Tasks

- Implement transaction categorization
- Implement budget tracking
- Implement spending summaries
- Implement dashboard metrics
- Implement alert rules
- Detect unusual transactions
- Create monthly financial summary
- Prepare charts and data for frontend

Main Deliverables

- Budget engine
- Alert engine
- Analytics API endpoints
- Spending category logic
- Dashboard metrics

Student 5 — Security, Testing, DevOps & Documentation Lead

Main Responsibility

Ensure the system is secure, tested, deployable, and well documented.

Tasks

- Review authentication and authorization
- Add role-based access for user/admin
- Write unit tests
- Write integration tests
- Create Postman collection
- Set up Docker
- Set up GitHub workflow, optional
- Prepare deployment environment

- Write user guide and technical documentation
- Help prepare final presentation

Main Deliverables

- Test suite
- Docker setup
- Deployment guide
- User documentation
- Final presentation support

---

8. Collaboration Plan

First Team Meeting Agenda

Use this agenda to get everyone aligned.

1. Agree on the Project Vision

Discuss and agree:

- What problem are we solving?
- Who is the target user?
- What makes this a FinTech project?
- What is the minimum product we must finish?

2. Agree on the Core Features

Must-have features:

- User registration and login
- Wallet creation
- Simulated deposit
- Transfer between users
- Double-entry ledger
- Transaction history
- Basic dashboard

Nice-to-have features:

- Budgets
- Alerts
- Admin panel
- Anomaly detection
- Recurring payment detection

- Transaction reversal

3. Agree on Technology Stack

Decide:

- Backend framework
- Frontend framework
- Database
- Hosting/deployment option
- Source control workflow

4. Assign Roles

Assign each student a clear area of ownership.

Each student should still help others, but one person should be responsible for each area.

5. Define the MVP

The MVP is the smallest version that can be demonstrated successfully.

Recommended MVP:

- User can register and log in
- User can create a wallet
- User can deposit simulated funds
- User can transfer funds to another user
- Ledger entries are created correctly
- User can view balance and transaction history

6. Define Weekly Milestones

Agree on what should be completed each week.

---

9. Suggested Milestones

Week 1 — Planning and Setup

Goals

- Finalize project scope
- Choose technology stack
- Create GitHub repository
- Design database schema
- Create wireframes

- Set up backend and frontend projects

Deliverables

- Project plan
- ERD draft
- UI wireframes
- Empty backend project
- Empty frontend project
- Initial README

Week 2 — Core Backend and Database

Goals

- Implement authentication
- Implement database migrations
- Implement wallet model
- Implement transaction model
- Implement ledger posting model

Deliverables

- User registration/login API
- Wallet API
- Database schema
- Ledger tables
- Seed data

Week 3 — Transfers and Ledger Logic

Goals

- Implement simulated deposits
- Implement wallet-to-wallet transfers
- Implement double-entry posting validation
- Implement balance updates
- Implement transaction history

Deliverables

- Deposit API
- Transfer API
- Transaction history API
- Ledger validation
- Balance updates

Week 4 — Frontend Integration

Goals

- Connect frontend to backend
- Implement login screen
- Implement dashboard
- Implement wallet page
- Implement transfer page
- Implement transaction history page

Deliverables

- Working user flow
- End-to-end transfer demo
- Basic dashboard

Week 5 — Analytics, Budgets and Alerts

Goals

- Add spending categories
- Add budget tracking
- Add dashboard charts
- Add alert rules
- Add admin overview, optional

Deliverables

- Budget page
- Analytics dashboard
- Alert system
- Admin dashboard, optional

Week 6 — Testing, Polish and Presentation

Goals

- Fix bugs
- Improve UI
- Add tests
- Finalize documentation
- Prepare demo script
- Prepare presentation

Deliverables

- Final working application
- Test results
- User guide
- Technical documentation
- Final presentation
- Demo script

---

10. Suggested Database Model

Main Tables

Users

Stores application users.

Fields:

- UserId
- FullName
- Email
- PasswordHash
- Role
- CreatedAt

Wallets

Stores each user wallet.

Fields:

- WalletId
- UserId

- WalletName
- Currency
- Balance
- Status
- CreatedAt

Transactions

Stores high-level financial transactions.

Fields:

- TransactionId
- TransactionType
- Amount
- Currency
- Status
- Description
- CreatedAt
- CreatedByUserId

Postings

Stores debit and credit ledger entries.

Fields:

- PostingId
- TransactionId
- WalletId
- DebitAmount
- CreditAmount
- CreatedAt

Budgets

Stores user budgets.

Fields:

- BudgetId
- UserId
- Category
- MonthlyLimit
- Month

- Year

Alerts

Stores system alerts.

Fields:

- AlertId
- UserId
- TransactionId
- AlertType
- Message
- Severity
- IsRead
- CreatedAt

---

11. API Endpoint Ideas

Authentication

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
```

Wallets

```
GET    /api/wallets
POST   /api/wallets
GET    /api/wallets/{id}
POST   /api/wallets/{id}/deposit
```

Transfers

```
POST /api/transfers
GET  /api/transfers/{id}
```

Transactions

```
GET /api/transactions
GET /api/transactions/{id}
```

Budgets

```
GET  /api/budgets
POST /api/budgets
PUT  /api/budgets/{id}
```

Analytics

```
GET /api/analytics/summary
GET /api/analytics/spending-by-category
GET /api/analytics/monthly-trend
```

Alerts

```
GET  /api/alerts
POST /api/alerts/{id}/mark-as-read
```

Admin

```
GET  /api/admin/users
GET  /api/admin/transactions
GET  /api/admin/alerts
POST /api/admin/wallets/{id}/freeze
```

---

12. MVP Scope

The team should not try to build everything at once.

The recommended MVP is:

1. User registration and login
2. Wallet creation
3. Simulated deposit
4. Transfer to another user
5. Double-entry ledger postings
6. Balance calculation
7. Transaction history
8. Basic dashboard

Once the MVP is working, add:

1. Budgets
2. Analytics
3. Alerts
4. Admin panel

5. Testing and deployment polish

---

13. Demo Script

For the final presentation, the team can show the following scenario:

1. Register Alice.
2. Register Bob.
3. Alice creates a Main Wallet.
4. Bob creates a Main Wallet.
5. Alice deposits €500 simulated funds.
6. Alice sends €50 to Bob.
7. Alice sees her balance decrease.
8. Bob sees his balance increase.
9. The system shows transaction history.
10. The team opens the ledger view and shows debit/credit entries.
11. The dashboard shows Alice's spending analytics.
12. The alerts page shows any generated alerts.
13. The admin panel shows all transactions.

This demo clearly shows the FinTech value of the project.

---

14. Project Risks and How to Manage Them

Risk 1 — Scope Too Large

Solution

Focus on the MVP first. Add advanced features only after the transfer and ledger flow works.

Risk 2 — Ledger Logic Is Confusing

Solution

Start with simple cases:

- Deposit
- Transfer
- Reversal, optional

Keep debit and credit rules simple and document them clearly.

Risk 3 — Frontend and Backend Are Not Aligned

Solution

Agree on API contracts early. Use Swagger or a shared API document.

Risk 4 — Team Members Work in Isolation

Solution

Have short weekly sync meetings. Each person should show what they completed and what they need from others.

Risk 5 — No Working Demo at the End

Solution

Make the end-to-end MVP work by Week 4. Use Weeks 5 and 6 for improvements.

---

15. Definition of Done

A feature is done only when:

- The backend logic works
- The database stores the correct data
- The frontend can use it
- Errors are handled properly
- It has been tested manually
- It is documented briefly
- It works in the demo environment

---

16. Final Deliverables

The final project should include:

- Source code repository
- Working deployed application or local Docker setup
- Database schema and ERD
- API documentation
- User guide
- Technical documentation
- Test evidence
- Final presentation
- Demo script

17. Suggested Repository Structure

```
fintrack/
  backend/
    src/
    tests/
    README.md
  frontend/
    src/
    public/
    README.md
  docs/
    project-plan.md
    api-design.md
    database-design.md
    demo-script.md
  docker-compose.yml
  README.md
```

18. Team Alignment Checklist

Use this checklist during the first meeting.

- [ ] We agree on the project name.
- [ ] We agree on the problem statement.
- [ ] We agree on the MVP features.
- [ ] We agree on the technology stack.
- [ ] We agree on each student's role.
- [ ] We agree on the database model.
- [ ] We agree on the API structure.
- [ ] We agree on the weekly milestones.
- [ ] We agree on how to use GitHub.
- [ ] We agree on the final demo scenario.

19. Recommended First Tasks

Backend Lead

- Create backend project
- Add authentication structure
- Add Swagger
- Create initial API endpoints

Database & Ledger Lead

- Design ERD
- Define ledger rules
- Create database migrations
- Create seed data

Frontend Lead

- Create wireframes
- Set up frontend project
- Create login and dashboard layout

Analytics Lead

- Define transaction categories
- Define budget rules
- Define alert rules
- Design dashboard metrics

Security, Testing & DevOps Lead

- Create testing plan
- Set up Docker
- Define GitHub workflow
- Create documentation structure

---

20. Summary

FinTrack is a strong FinTech student project because it is practical, realistic, and technically meaningful.

It gives the team a chance to demonstrate:

- Financial transaction processing
- Secure user management
- Ledger-based accounting
- Frontend dashboard design
- Data analytics
- Software architecture
- Testing and deployment

The key to success is to finish the MVP first, then add analytics, alerts, and admin features once the core wallet and ledger flow is stable.